

قيمته **MVP 15,000** رأيك ممتاز، وفيه بذرة منتج حقيقي، لا مجرد أداة توليد كود. سأحولك إلى حل تنفيذي متكامل لمشروع دولار، بحيث يكون قابلاً للبيع، قابلاً للعرض أمام عميل، وقابلاً للتوسعة لاحقاً.

الاسم المقترح للمنتج:

## SRS-to-Service Hukm Engine

بمنطق السبب، الشرط، المانع، SRS من Web Services منصة توليد الصحة، الفساد، البطلان، العزيمة، والرخصة

الفكرة التجارية ليست

”ويكتب كود SRS يقرأ AI“

بل:

إلى أحكام توليد مضبوطة: لا كود بلا سبب، لا توليد بلا شرط، لا تنفيذ مع مانع، لا قبول SRS منصة تحول الـ بلا صحة، ولا رخصة بلا إعلان وتوثيق.

هذا هو الفرق بين مشروع عادي ومشروع يستحق 15 ألف دولار

---

## نطاق مشروع الـ 15,000 دولار 1.

demo هذا نطاق ضخمة وسيفشل أو يصبح. ”SRS هو“ منصة تولد أي نظام كامل من أي K لا أنصح أن يكون نطاق الـ 15 كاذباً.

النطاق الصحيح هو:

ثابتة Templates، واحد Stack باستخدام، SRS من Backend Web Service يولد MVP، كامل Traceability واضح، وتقرير Validation.

## MVP ما يدخل في نطاق الـ

بصيغة SRS النظام يأخذ

Markdown / Text / JSON

Optional: PDF أو DOCX للاستخراج بدون OCR إذا كان النص قابلاً للاستخراج

ثم ينتج:

1. Requirement IR

2. Cause / Condition / Blocker model
3. OpenAPI contract
4. Laravel أو FastAPI REST API backend
5. Database migrations
6. Request validation
7. Service layer
8. Tests
9. Docker-ready project
10. METHODOLOGY\_AUDIT.md
11. Traceability matrix

## MVP ما لا يدخل في نطاق الـ

حتى لا يتمدد المشروع بلا ضبط

- كامل Frontend لا توليد
- لا Mobile apps
- لا Microservices
- لا دعم كل اللغات
- تلقائي Production لا نشر
- الممسوحة PDFs للـ OCR لا
- لا تكاملات حكومية أو بنكية حقيقية
- ضعيف سينتج نظامًا صحيحًا SRS لا ضمان أن أي

”AI مضبوط ومثبت بالدليل، وليس“ ساحر Backend Service هو محرك توليد K المنتج الذي يستحق 15

---

## 2. الاختيار التقني الأفضل

أقترح هذا التقسيم، Laravel و Templates لأنك تفكر بمنطق

Control Plane:

- Next.js frontend
- FastAPI orchestration backend
- PostgreSQL

AI Layer:

- OpenAI Structured Outputs
- خاص orchestrator أو Agents SDK
- لاحقًا LangGraph Optional

Generated Service:

- Laravel REST API



## Analyze SRS

فخرج له:

Requirements extracted: 18  
Valid requirements: 13  
Ambiguous requirements: 3  
Blocked requirements: 2  
Suggested generated modules:  
- Auth  
- Users  
- Applications  
- Documents  
- Reviews  
- Notifications

بعد الموافقة:

## Generate Service

فنتنتج المنصة:

- Git repository ZIP
- OpenAPI contract
- Laravel backend
- Tests
- Docker compose
- Methodology audit report
- Requirement traceability matrix

هذه تجربة قابلة للبيع.

---

## 4. قلب النظام: Requirement Hukm IR

للمتطلب الواحد JSON Schema أهم شيء في المشروع كله هو الـ

الخاص بك "IR" أو "الـ" "AST" هذا هو "الـ"

### RequirementHukmIR

```
{  
  "id": "REQ-APP-001",
```

```

"source": {
  "document": "SRS-v1.md",
  "section": "4.2.1",
  "quote": "The applicant shall submit a membership application."
},
"statement": "Applicant can submit a membership application.",
"actor": "Applicant",
"sabab": {
  "type": "documented_requirement",
  "description": "Applicant initiates application submission from SRS section 4.2.1"
},
"conditions": [
  {
    "id": "COND-AUTH-001",
    "description": "Applicant must be authenticated",
    "check_type": "auth",
    "required": true
  },
  {
    "id": "COND-FORM-001",
    "description": "Application form must be complete",
    "check_type": "validation",
    "required": true
  }
],
"blockers": [
  {
    "id": "MANI-DUP-001",
    "description": "Applicant already has an active application",
    "severity": "business_critical",
    "resolution": "Reject submission with clear error"
  },
  {
    "id": "MANI-SUSP-001",
    "description": "Applicant account is suspended",
    "severity": "security_critical",
    "resolution": "Return forbidden response and audit the attempt"
  }
],
"effects": [
  {
    "type": "state_change",
    "description": "Application status becomes Submitted"
  },
  {
    "type": "audit",
    "description": "Submission action is recorded"
  }
],

```

```

{
  "type": "notification",
  "description": "Applicant receives confirmation"
},
"template_target": {
  "framework": "laravel",
  "module": "applications",
  "artifacts": [
    "controller",
    "service",
    "request_validation",
    "migration",
    "model",
    "policy",
    "tests"
  ]
},
"hukm": {
  "mode": "azimah",
  "rukhsa": null
},
"acceptance_tests": [
  "Authenticated applicant can submit complete application",
  "Unauthenticated user cannot submit application",
  "Duplicate active application is rejected",
  "Suspended applicant is rejected",
  "Audit log is created"
],
"invalidity_rules": {
  "fasad": [
    "Notification missing but application state is correct",
    "Audit metadata incomplete",
    "Error message not descriptive"
  ],
  "batil": [
    "Unauthenticated user can submit",
    "Application accepted despite duplicate active application",
    "Generated endpoint has no SRS source"
  ]
}
}

```

هو الذي يعطيك IR هذا الـ

Traceability

Validation

Code generation  
Test generation  
Audit report  
Repair vs regeneration decision

”يكتب كود AI“ بدونهُ ستعود إلى

---

## 5. المنصة Database Design الـ

المنصة نفسها تحتاج قاعدة بيانات لتخزين المشاريع، المتطلبات، الأحكام، دورات التوليد، والتقارير.

### الجدول الأساسية

projects

- id
- name
- target\_stack
- policy\_pack
- mode: azimah | rukhsa
- status
- created\_at

srs\_documents

- id
- project\_id
- filename
- content\_text
- parsed\_at

requirements

- id
- project\_id
- requirement\_code
- source\_section
- source\_quote
- statement
- actor
- status: extracted | valid | ambiguous | blocked | generated | accepted | rejected

causes

- id
- requirement\_id
- sabab\_type

- description

#### conditions

- id
- requirement\_id
- code
- description
- check\_type
- required
- status: met | missing | failed

#### blockers

- id
- requirement\_id
- code
- description
- severity
- status: absent | present
- resolution\_path

#### effects

- id
- requirement\_id
- effect\_type
- description
- target\_artifact

#### generation\_runs

- id
- project\_id
- run\_number
- mode
- status
- started\_at
- completed\_at

#### generated\_artifacts

- id
- generation\_run\_id
- requirement\_id
- file\_path
- artifact\_type
- content\_hash
- status

#### validation\_results

- id
- generation\_run\_id

- requirement\_id
- verdict: sahih | fasid | batil
- reason
- evidence

rukhsa\_records

- id
- project\_id
- requirement\_id
- justification
- waived\_rules
- non\_waivable\_rules
- approved\_by
- expires\_condition

audit\_entries

- id
- project\_id
- requirement\_id
- event\_type
- message
- metadata\_json
- created\_at

النقطة الذهبية هنا

كل ملف كود يجب أن يرجع إلى `requirement_id`.

يصبح requirement بذلك أي كود بلا

orphan code = batil

---

## خط الأنابيب التنفيذي 6.

الكامل pipeline هذا هو الـ

1. SRS Upload
- ↓
2. SRS Parser Agent
- ↓
3. Requirement Extraction
- ↓
4. Hukm IR Builder
- ↓

5. Condition Checker
- ↓
6. Blocker Detector
- ↓
7. Architecture Mapper
- ↓
8. OpenAPI Contract Generator
- ↓
9. Template-based Code Generator
- ↓
10. Test Generator
- ↓
11. Sandbox Build Runner
- ↓
12. Validator Agent
- ↓
13. Fasad/Batil Classifier
- ↓
14. Repair Loop or Regeneration
- ↓
15. Audit Agent
- ↓
16. Export Service

القاعدة:

sabab exists  
AND shurut are met  
AND mani3 is absent  
THEN generation is allowed

لكن بعد التوليد:

generated code  
MUST satisfy tests  
MUST map to requirement  
MUST pass security checks  
MUST pass architecture rules  
MUST produce audit evidence

---

## 7. المطلوبة في المشروع Agents

وممنوعات، output، input، له وظيفة Agent حرة. كل agents لا تجعلها

## 7.1 SRS Parser Agent — المستقرئ

### وظيفته

ويستخرج المتطلبات الأولية SRS يقرأ

### Input

SRS text  
Project context  
Target stack  
Policy pack

### Output

```
[  
  {  
    "id": "REQ-AUTH-001",  
    "statement": "Users shall login using email and password",  
    "source_section": "4.1.2",  
    "actor": "User",  
    "confidence": 0.91  
  }  
]
```

### ممنوع عليه

- لا يولد كود
- لا يقرر الصحة
- غير مذكورة requirements لا يخترع

---

## 7.2 Condition Checker — المحقق

### وظيفته

يفحص هل المتطلب قابل للتوليد

### أمثلة شروط

- واضح source للمتطلب
- actor له
- expected effect له
- لا يناقض متطلباً آخر
- target module له

- acceptance criteria له

## Output

```
{  
  "requirement_id": "REQ-AUTH-001",  
  "conditions_status": "passed",  
  "missing_conditions": []  
}
```

لو الشرط ناقص

```
{  
  "requirement_id": "REQ-PERF-001",  
  "conditions_status": "failed",  
  "missing_conditions": [  
    "Performance threshold is not measurable",  
    "Target endpoint is not specified"  
  ]  
}
```

الحكم:

لا توليد

---

## 7.3 Blocker Detector — كاشف الموانع

### وظيفته

يفحص الموانع قبل التوليد

### أنواع الموانع

Security blocker  
Architecture blocker  
Dependency blocker  
Compliance blocker  
Data privacy blocker  
Contradiction blocker

### مثال

```
{
```

```

"requirement_id": "REQ-AUTH-004",
"blocker": {
  "type": "security",
  "description": "SRS requests storing passwords in plain text",
  "severity": "critical",
  "resolution": "Replace with password hashing policy"
},
"decision": "halt_generation"
}

```

أن يقول: "سأطبقها بطريقة آمنة" دون تعديل المتطلب أو تسجيل استثناء. هذا يمنع الالتفاف الصامت AI لا يسمح للـ

---

## 7.4 Architecture Mapper Agent — مفتي البنية

### وظيفته

artifacts يحوّل المتطلب إلى

مثلاً:

REQ-APP-001

يتحول إلى:

```

ApplicationController
ApplicationService
Application model
CreateApplicationRequest
SubmitApplicationRequest
applications migration
ApplicationPolicy
ApplicationFeatureTest

```

### Output

```

{
  "requirement_id": "REQ-APP-001",
  "artifacts": [
    {
      "type": "controller",
      "path": "app/Http/Controllers/ApplicationController.php"
    },
    {

```

```

    "type": "service",
    "path": "app/Services/ApplicationService.php"
  },
  {
    "type": "test",
    "path": "tests/Feature/ApplicationSubmitTest.php"
  }
]
}

```

---

## 7.5 Generator Agent — المولّد

### وظيفته

فقط Templates يملأ.

### ممنوع عليه

- architecture لا يغيّر
- requirement بلا endpoint لا يضيف
- AuditLog لا يحذف
- RBAC لا يتجاوز
- secrets لا يكتب
- غير ضروري test لا يقرر أن

slots. المولّد ليس حرًا. هو يشغل داخل

---

## 7.6 Validator Agent — المدقق

### وظيفته

يحكم على الناتج:

sahih  
fasid  
batil

لكن لا يعتمد على رأيه فقط. يعتمد على أدلة:

- tests
- static analysis
- file existence

- OpenAPI diff
  - requirement mapping
  - security rules
  - template compliance
- 

## 7.7 Repair Agent — المصحح

يدخل فقط إذا الحكم

fasid

مثال:

ناقص AuditLog الكود صحيح وظيفيًا لكن

هنا يطلب منه

service logic ولا endpoint فقط، لا تغيّر AuditLog أضف

أما إذا الحكم

batil

فالمصحح لا يعمل. يتم إهدار الناتج وإعادة التوليد من المرحلة المناسبة

---

## 7.8 Audit Agent — الموثق

ينتج:

METHODOLOGY\_AUDIT.md  
TRACEABILITY\_MATRIX.md  
VALIDATION\_REPORT.json  
RUKHSA\_DECLARATIONS.md

وهذا جزء مهم من قيمة المنتج

---

## 8. Templates: **تقييد الاجتهاد**

هيكل القوالب المقترح:

```
templates/
├── laravel-api/
│   ├── controller.tpl.php
│   ├── service.tpl.php
│   ├── model.tpl.php
│   ├── migration.tpl.php
│   ├── request-store.tpl.php
│   ├── request-update.tpl.php
│   ├── policy.tpl.php
│   ├── feature-test.tpl.php
│   ├── routes.tpl.php
│   └── openapi-path.tpl.yaml
├── security/
│   ├── auth-middleware.tpl.php
│   ├── rbac-check.tpl.php
│   └── audit-log.tpl.php
├── docker/
│   ├── Dockerfile.tpl
│   └── docker-compose.tpl.yml
└── docs/
    ├── methodology-audit.tpl.md
    └── traceability-matrix.tpl.md
```

مثال Controller Template:

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests\{{ StoreRequestClass }};
use App\Services\{{ ServiceClass }};
use App\Support\AuditLog;

class {{ ControllerClass }} extends Controller
{
    public function store({{ StoreRequestClass }} $request, {{ ServiceClass }} $service)
    {
        $this->authorize('{{ ability }}', {{ ModelClass }}::class);

        $result = $service->create(
            user: $request->user(),
            data: $request->validated()
        );
    }
}
```

```

);

AuditLog::record(
  actor: $request->user(),
  action: '{{ audit_action }}',
  subject: $result,
  requirementId: '{{ requirement_id }}'
);

return response()->json($result, 201);
}
}

```

لاحظ أن:

AuditLog  
 Authorization  
 requirementId  
 validated request  
 service layer

ليست اختيارية. هي داخل القالب

لا "يتذكرها". القالب يفرضها AI الـ

## Policy File نموذج العزيمة والرخصة في 9.

أنت تحتاج ملف سياسة واضح

### policy.default.yml

policy\_name: default\_secure\_api

azimah:

required:

- requirement\_id
- source\_reference
- openapi\_contract
- controller
- service\_layer
- request\_validation
- authorization
- audit\_log\_for\_mutations

- feature\_tests
- docker\_build
- traceability\_record

quality\_gates:

- tests\_must\_pass: true
- min\_test\_coverage: 80
- static\_analysis\_required: true
- security\_scan\_required: true
- no\_orphan\_code: true
- no\_hallucinated\_endpoints: true

rukhsa:

allowed\_only\_in:

- prototype
- internal\_demo
- sandbox

requires:

- explicit\_justification
- waived\_rules\_list
- expiry\_condition
- audit\_record

waivable:

- min\_test\_coverage
- notification\_provider\_integration
- external\_payment\_integration

non\_waivable:

- authentication
- authorization
- password\_hashing
- input\_validation
- secrets\_handling
- audit\_for\_sensitive\_mutations
- no\_orphan\_code
- no\_plaintext\_passwords

هذا الملف يحول العزيمة والرخصة إلى سياسة قابلة للتنفيذ.

---

## تصنيف الصحة والفساد والبطلان. 10

## Decision Matrix

| الحالة | المعنى الهندسي                                     | القرار             |
|--------|----------------------------------------------------|--------------------|
| صحيح   | يطابق المتطلب، يمر بالاختبارات، لا يخالف السياسات  | قبول               |
| فاسد   | الأصل صحيح لكن الوصف ناقص أو قابل للإصلاح          | Repair loop        |
| باطل   | الأصل غير صحيح أو لا سبب له أو يخالف شرطاً جوهرياً | إهدار وإعادة توليد |

## أمثلة عملية

| الحالة                                                         | التصنيف                 | السبب               |
|----------------------------------------------------------------|-------------------------|---------------------|
| Endpoint موجود، tests تمر، audit موجود                         | صحيح                    | مستوفٍ              |
| Endpoint notification صحيح لكن Endpoint ناقصة                  | فاسد                    | أثر ثانوي ناقص      |
| Controller architecture مخالفاً DB يتصل مباشرة بالـ Controller | policy فاسد أو باطل حسب | وصف معماري معتل     |
| Endpoint SRS source مولد بلا Endpoint                          | باطل                    | لا سبب              |
| Login password plaintext يخزن                                  | باطل                    | مانع أمني جوهري     |
| Submit application duplicate active يقبل application           | باطل                    | خالف مانعاً منصوصاً |
| Test coverage 70% في demo معلن برخصة                           | صحيح برخصة              | الرخصة موثقة        |
| Test coverage 70% في azimah                                    | فاسد                    | قابل للإصلاح        |

## 11. Validation والـ Sandbox

Sandbox كل مشروع مولد يجب أن يبنى داخل

generated-project/

↓

docker build

↓

composer install

↓

php artisan test

↓

static analysis

↓

security scan

↓

OpenAPI validation

↓

audit report

يشرح أن Docker. ويجب عزله عن منصة التحكم، dependencies، لأن التوليد سيشغل كودًا و Docker استخدام كطبقة عزل أساسية، لكن يجب التعامل أيضًا مع سطح هجوم control groups و namespaces تستخدم containers Docker daemon وإعدادات الحاويات ([Docker Documentation](#))

وكذلك قواعد، secrets scanning و SCA و SAST لأنه يدعم pipeline في الـ Semgrep للتحقق الأمني، يمكن وضع linters، security مناسب لتشغيل CI، GitHub Actions و ([Semgrep](#)). coding standards مخصصة checks، coverage، functional tests، custom checks و PR. ([GitHub Docs](#)) أو commit على كل

---

## 12. OpenAPI Contract First

قبل الكود، تولد المنصة

openapi.yaml

مثال:

paths:

/applications:

post:

summary: Create application

operationId: createApplication

x-requirement-id: REQ-APP-001

security:

- bearerAuth: []

requestBody:

required: true

responses:

"201":

description: Application created

"401":

description: Unauthenticated

"403":

description: Forbidden

"422":

description: Validation error

وجود:

x-requirement-id

نفسه مرتبطًا بالسبب API مهم جدًا؛ لأنه يجعل الـ

client ؛ توثيقه يذكر أنه يدعم عشرات server stubs أو clients لتوليد OpenAPI Generator ويمكن لاحقًا استخدام generators server stubs، [openapi-generator.tech](https://openapi-generator.tech) وأن القوالب يمكن استبدالها بقوالب مخصصة،

## 13. واجهات المنصة

### 1 شاشة: Projects

- Project name
- Target stack
- Mode: Azimah / Rukhsa
- Policy pack
- Status

### 2 شاشة: SRS Upload

- Upload file
- Paste text
- Parse SRS
- Show extracted sections

### 3 شاشة: Requirement Review

جدول:

| ID           | Statement                | Source | Actor  | Status    | Action       |
|--------------|--------------------------|--------|--------|-----------|--------------|
| REQ-AUTH-001 | User login               | §4.1   | User   | Valid     | Approve      |
| REQ-PERF-001 | System is fast           | §8.1   | N/A    | Ambiguous | Clarify      |
| REQ-SEC-002  | Store password plaintext | §5.3   | System | Blocked   | View blocker |



```

|   |   | Models/
|   |   | Policies/
|   |   | Services/
|   |   | Support/
|   |   |   | AuditLog.php
|   | database/
|   |   | migrations/
|   | routes/
|   |   | api.php
|   | tests/
|   |   | Feature/
|   | docs/
|   |   | openapi.yaml
|   |   | TRACEABILITY_MATRIX.md
|   |   | METHODOLOGY_AUDIT.md
|   |   | VALIDATION_REPORT.json
|   | docker-compose.yml
|   | Dockerfile
|   | README.md

```

header يحتوي generated file كل:

```

/**
 * Generated by SRS-to-Service Hukm Engine
 * Requirement: REQ-APP-001
 * Source: SRS §4.2.1
 * Hukm: Azimah
 * Verdict: Sahih
 */

```

.هذا وحده يرفع قيمة المنتج لأنك تتبع قابلية التدقيق لا مجرد كود

## 15. METHODOLOGY\_AUDIT.md نموذج

مثال التقرير:

```
# Methodology Audit Report
```

```
## Project
Membership Service
```

```
## Generation Mode
Azimah
```

## Requirement REQ-APP-001

### Source

SRS §4.2.1:

"The applicant shall submit a membership application."

### Sabab

Documented requirement to submit an application.

### Conditions

- Applicant must be authenticated: PASSED
- Application form must be complete: PASSED
- Required documents must exist: PASSED

### Mani'

- Duplicate active application: ABSENT
- Suspended account: ABSENT
- Closed submission period: ABSENT

### Generated Artifacts

- app/Http/Controllers/ApplicationController.php
- app/Services/ApplicationService.php
- app/Http/Requests/SubmitApplicationRequest.php
- tests/Feature/SubmitApplicationTest.php
- docs/openapi.yaml

### Validation

- Feature tests: PASSED
- Static analysis: PASSED
- Security scan: PASSED
- OpenAPI mapping: PASSED

### Hukm

Sahih

### Effect

Accepted into generated service.

هذا التقرير هو "الدليل الشرعي/الهندسي" على صحة التوليد.

---

## 16. الخاصة بالمنصة API الـ

## Projects

POST /projects  
GET /projects/{id}

## SRS

POST /projects/{id}/srs  
POST /projects/{id}/parse

## Requirements

GET /projects/{id}/requirements  
GET /requirements/{id}  
PATCH /requirements/{id}/approve  
PATCH /requirements/{id}/mark-ambiguous

## Generation

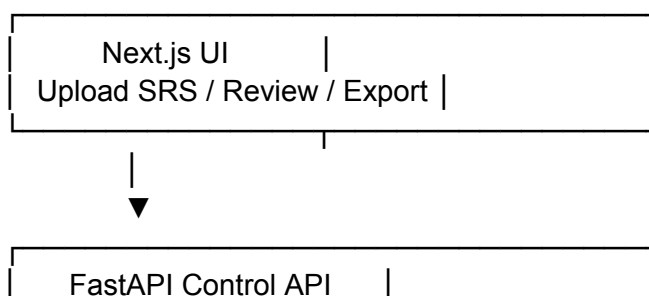
POST /projects/{id}/generation-runs  
GET /generation-runs/{id}  
POST /generation-runs/{id}/repair  
POST /generation-runs/{id}/export

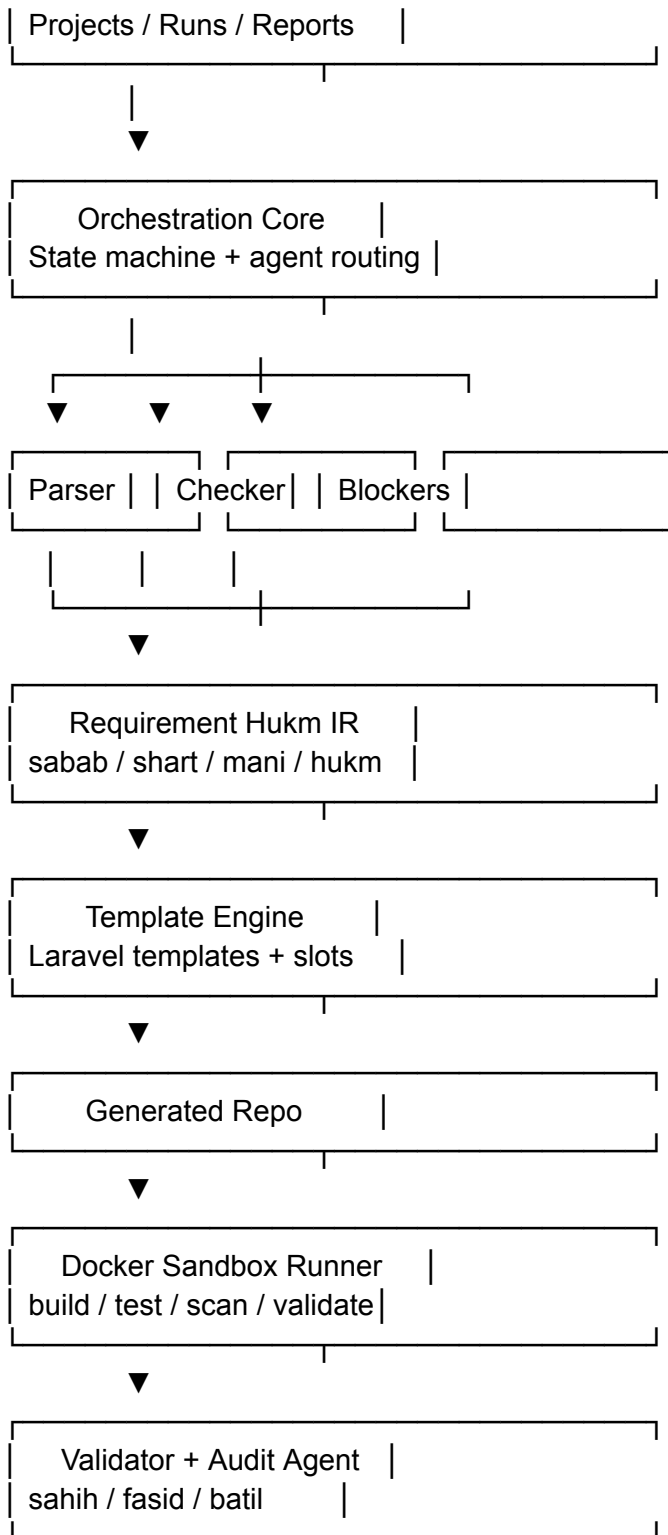
## Reports

GET /projects/{id}/traceability  
GET /projects/{id}/audit  
GET /projects/{id}/validation-report

---

# 17. Architecture Diagram





## تقسيم ميزانية الـ 15,000 دولار 18.

احترافي MVP هذا تقسيم منطقي يبيع المشروع كـ

| البند                                          | القيمة          |
|------------------------------------------------|-----------------|
| Product architecture + Hukm methodology design | \$1,500         |
| RequirementHukmlR JSON Schema + DB model       | \$1,500         |
| Next.js dashboard basic UI                     | \$2,000         |
| FastAPI control backend                        | \$2,000         |
| SRS parser + structured extraction             | \$1,800         |
| Agents / orchestration state machine           | \$2,200         |
| Laravel template generator                     | \$2,400         |
| Sandbox build/test/validation runner           | \$1,000         |
| Audit + traceability reports                   | \$900           |
| Demo SRS + generated sample service            | \$700           |
| <b>Total</b>                                   | <b>\$15,000</b> |

:هذا التقسيم عادل لأن العميل لا يدفع مقابل "كود فقط"، بل مقابل

منهجية

منصة

AI workflow

قوالب

تحقق

تقارير

منتج قابل للعرض

## أهم مخرجات التسليم. 19.

:في نهاية المشروع، يجب أن تكون المخرجات

1. Running web dashboard
2. Working SRS parser
3. RequirementHukmlR schema
4. PostgreSQL schema
5. Laravel code generator
6. 8-12 production-style templates
7. Sandbox runner
8. Test execution report

9. Fasad/Batil classifier
10. METHODOLOGY\_AUDIT.md generator
11. Traceability matrix generator
12. One complete sample generated service
13. Documentation for extending templates
14. Demo script for sales presentation

يجب أن يكون مثل demo sample الـ

## Membership Application Service

ويحتوي:

- Auth
- Applicants
- Applications
- Documents
- Reviews
- Audit logs

هذا قريب من أمثلة حكومية/مؤسسية ويخدم قصة البيع

---

## 20. Sample Demo Flow أمام عميل

السيناريو البيعي:

1. بسيط لنظام طلبات انتساب SRS نرفع.
2. requirement المنصة تستخرج 14.
3. Requirement ان يظهر ان ambiguous.
4. Requirement لأنه يخالف blocked واحد.
5. requirement نوافق على 11.
6. OpenAPI contract المنصة تولد.
7. Laravel backend المنصة تولد.
8. tests المنصة تشغل.
9. ناقصة notification لأن Fasid يظهر متطلب واحد.
10. Repair Agent يصلحه.
11. Sahih: المنصة تنتج الحكم النهائي.
12. فيه الخدمة والتقارير ZIP نحمل.
13. ونرى METHODOLOGY\_AUDIT.md نفتح.
- السبب، الشرط، المانع، الحكم، الأثر

هذا عرض قوي جدًا

---

## كيف تجعل المشروع لا يتحول إلى فوضى؟ 21.

“ضع هذه القواعد كـ”دستور المنصة“:

1. requirement\_id لا كود بلا.
2. requirement لا source.
3. لا توليد قبل تحقق الشروط.
4. لا توليد مع وجود مانع.
5. OpenAPI path بلا endpoint لا.
6. AuditLog بلا mutation لا.
7. Authorization بلا sensitive endpoint لا.
8. لا قبول لكود لم يمر بالاختبارات.
9. justification لا رخصة بلا.
10. Repair لا للبطل.
11. Regeneration لا للإصلاح.
12. agent لا يحكم على عمله بنفسه.

هذه القواعد يجب أن تكون في:

POLICY.md  
policy.default.yml  
system prompts  
validator code  
audit report

---

## وأي لا يستخدم؟ AI أين يستخدم 22.

### في AI يستخدم

- requirements استخراج من SRS
- كشف الغموض
- conditions اقتراح
- blockers اقتراح
- داخل القالب business logic توليد
- tests توليد
- شرح أسباب الفساد أو البطلان
- إصلاح الفساد
- documentation توليد

## وحده في AI لا يستخدم

- قبول الكود النهائي
- security تجاوز
- إسقاط شروط العزيمة
- منح الرخصة
- deployment production تشغيل
- security scan تقرير أن غير مهم

القاعدة:

الذكاء الاصطناعي شاهد ومجتهد وموّد، لكنه ليس القاضي النهائي

القاضي النهائي هو:

Policy + Tests + Static Analysis + Traceability + Human Review

---

## خوارزمية الحكم. 23.

```
def decide_generation(requirement):
    if not requirement.source:
        return "reject", "No sabab: requirement has no SRS source"

    if not all_conditions_met(requirement):
        return "reject", "Missing shurut"

    blocker = detect_blocker(requirement)
    if blocker:
        return "halt", blocker.resolution_path

    return "generate", "Allowed"

def validate_artifact(requirement, artifacts, evidence):
    if has_orphan_code(artifacts):
        return "batil", "Generated code without sabab"

    if violates_non_waivable_security(requirement, artifacts):
        return "batil", "Non-waivable security violation"

    if not satisfies_core_effect(requirement, evidence):
        return "batil", "Core effect not achieved"
```

```
if has_repairable_defects(requirement, evidence):  
    return "fasid", "Repairable defect"
```

```
return "sahih", "All checks passed"
```

في العرض centerpiece هذه الخوارزمية وحدها تصلح أن تكون

---

## كيف تتعامل مع الرخصة تجاريًا؟ 24.

demo. الرخصة مهمة جدًا لأنها تمنع كذب الـ

:عندما يختار المستخدم

Mode: Rukhsa

:يجب أن تظهر له شاشة

You are generating under Rukhsa mode.

This means selected rules may be relaxed for prototype/demo only.

Non-waivable security rules remain enforced.

This will be recorded in METHODOLOGY\_AUDIT.md.

:ثم يكتب

Justification:

Internal prototype for stakeholder demo. Not for production.

:ثم يحدد

Waive:

- Minimum 80% test coverage
- Real notification provider integration

:لكن النظام يمنع إسقاط

- authentication
- authorization
- validation
- audit for sensitive actions
- password hashing

في التقرير:

## ## Rukhsa Declaration

This service was generated under Rukhsa mode.

Justification:

Internal prototype for stakeholder demo.

Waived:

- Minimum test coverage reduced from 80% to 40%
- Notification provider mocked

Non-waived:

- Authentication
- Authorization
- Input validation
- Audit logs
- Secrets handling

Production status:

Not production eligible until rukhsa is lifted.

هذه نقطة بيع كبيرة:

يصبح صادقًا ومؤطرًا **demo** حتى الـ

---

## الذي أنصح ببنائه أولاً MVP الـ 25.

النسخة الأولى يجب أن تدعم فقط:

Target stack:

Laravel REST API

Modules:

- Auth
- CRUD resource
- File upload metadata only
- Review workflow
- Audit logs

Input:

- SRS markdown/text

- Optional JSON structured SRS

Output:

- Laravel project ZIP
- OpenAPI YAML
- Tests
- Docker compose
- Audit docs

requirements: لا تبدأ بتوليد كل شيء. ابدأ بـ 4 أنواع

1. CRUD requirement
2. Auth/RBAC requirement
3. Workflow state transition requirement
4. Audit/notification requirement

هذه كافية لإظهار قوة الفكرة.

---

## 26. Backlog تفصيلي

### Epic 1: Project Setup

- Create Next.js dashboard
- Create FastAPI backend
- Configure PostgreSQL
- Add project CRUD
- Add generation run tracking

### Epic 2: SRS Ingestion

- Paste SRS text
- Upload markdown/text
- Extract sections
- Store original source
- Show parsed sections

### Epic 3: Requirement IR

- Define RequirementHukmIR schema
- Validate AI output against schema

- Store requirements
- Mark ambiguous requirements
- Manual approve/reject

## **Epic 4: Hukm Engine**

- Cause validator
- Condition checker
- Blocker detector
- Azimah/rukhsa policy engine
- Fasad/batil classifier

## **Epic 5: Code Generation**

- Template registry
- Laravel controller template
- Laravel service template
- Laravel request validation template
- Laravel migration template
- Laravel model template
- Laravel feature test template
- Route generator

## **Epic 6: OpenAPI**

- Generate openapi.yaml
- Add x-requirement-id
- Validate path mapping

## **Epic 7: Sandbox**

- Generate project workspace
- Build Docker image
- Run tests
- Capture logs
- Parse failures
- Store evidence

## **Epic 8: Audit**

- Generate TRACEABILITY\_MATRIX.md
- Generate METHODOLOGY\_AUDIT.md

- Generate VALIDATION\_REPORT.json
- Export ZIP

## 27. معايير القبول النهائية.

:لا تقبل المشروع إلا إذا حقق الآتي

1. جديد project يستطيع إنشاء.
2. نصي SRS يستطيع إدخال.
3. sample SRS من requirements + يستخرج 10.
4. واحد على الأقل requirement ambiguous يميز.
5. أممي واحد على الأقل blocker يكتشف.
6. requirements approved من Laravel backend يولد.
7. OpenAPI contract يولد.
8. tests يولد.
9. sandbox داخل tests يشغل.
10. result يصنف إلى Sahih/Fasid/Batil.
11. واحدة على الأقل Fasid يصلح حالة.
12. واحدة على الأقل Batil يرفض حالة.
13. METHODOLOGY\_AUDIT.md يولد.
14. TRACEABILITY\_MATRIX.md يولد.
15. قابل للتنشغيل ZIP يصدر.

محددة وغير فضفاضة K هذه المعايير تجعل الـ 15

## 28. المخاطر وكيف نغلقها

| الخطر                        | الحل                             |
|------------------------------|----------------------------------|
| AI requirements يخترع        | source quote لا قبول بلا         |
| AI traceability يولد كود بلا | requirement_id يحتاج artifact كل |
| SRS غامض                     | ولا توليد mark ambiguous         |
| Templates تصبح معقدة         | فقط template ابدأ بـ 8-12        |
| Repair loop يفسد الصحيح      | defect يعمل على Repair Agent     |
| Security demo يتم تجاوزه في  | non-waivable rules               |

المشروع يكبر جدًا

واحد ونوع واحد من الخدمات Stack

full platform العميل يتوقع

backend service generator MVP: عرض النطاق بوضوح

---

## القيمة التجارية. 29.

هذا المشروع يمكن بيعه بثلاث رسائل

### الرسالة الأولى

نحن لا نولد كودًا عشوائيًا

SRS. نحن نولد خدمة مرتبطة بكل متطلب في الـ

### الرسالة الثانية

له سبب، شروط، موانع، اختبارات، وتقرير تدقيق endpoint كل

### الرسالة الثالثة

المنصة تميز بين الكود القابل للإصلاح والكود الواجب رفضه

“AI software generator” هذه الرسائل أقوى من

---

## الصياغة التنفيذية النهائية. 30.

هو K الحل الكامل لمشروع 15

SRS-to-Service Hukm Engine باسم Web MVP منصة

Backend نصيًا، تستخرج المتطلبات كأسباب، تفحص شروط التوليد، تكشف الموانع الأمنية والمعمارية، تولد SRS تدخل تصنف الناتج إلى صحيح أو فاسد، Docker Sandbox تشغل الاختبارات داخل، Templates من Laravel REST API و Traceability و OpenAPI أو باطل، تصلح الفاسد فقط، ترفض الباطل، وتصدر خدمة قابلة للتشغيل مع METHODOLOGY\_AUDIT.md.

---

## القرار الأهم. 31.

Agents: ابدأ من هذا الملف، لا من الواجهة ولا من الـ

RequirementHukmlR.schema.json

لأنه هو أصل النظام

:بعده مباشرة

policy.default.yml

ثم:

template registry

ثم:

validator

ثم:

AI agents

:الترتيب الصحيح

Schema

↓

Policy

↓

Templates

↓

Validator

↓

Agents

↓

UI

.وليس العكس

# الخلاصة

مشروع الـ 15 ألف دولار يجب أن يكون

code generator ليس

governed code generation engine بل

بمنطق:

لا كود بلا سبب

لا توليد بلا شرط

لا تنفيذ مع مانع

لا قبول بلا صحة

الفاقد يُصلح

الباطل يُهدر

العزيمة هي الأصل

الرخصة استثناء موثق

والمنتج القابل للبيع هو

**AI-assisted SRS-to-Laravel-Service Generator with Hukm Governance,  
Template Constraints, Sandbox Validation, and Full Traceability.**

هذا حل قوي، مركز، قابل للتنفيذ، وقيمتة التجارية واضحة.